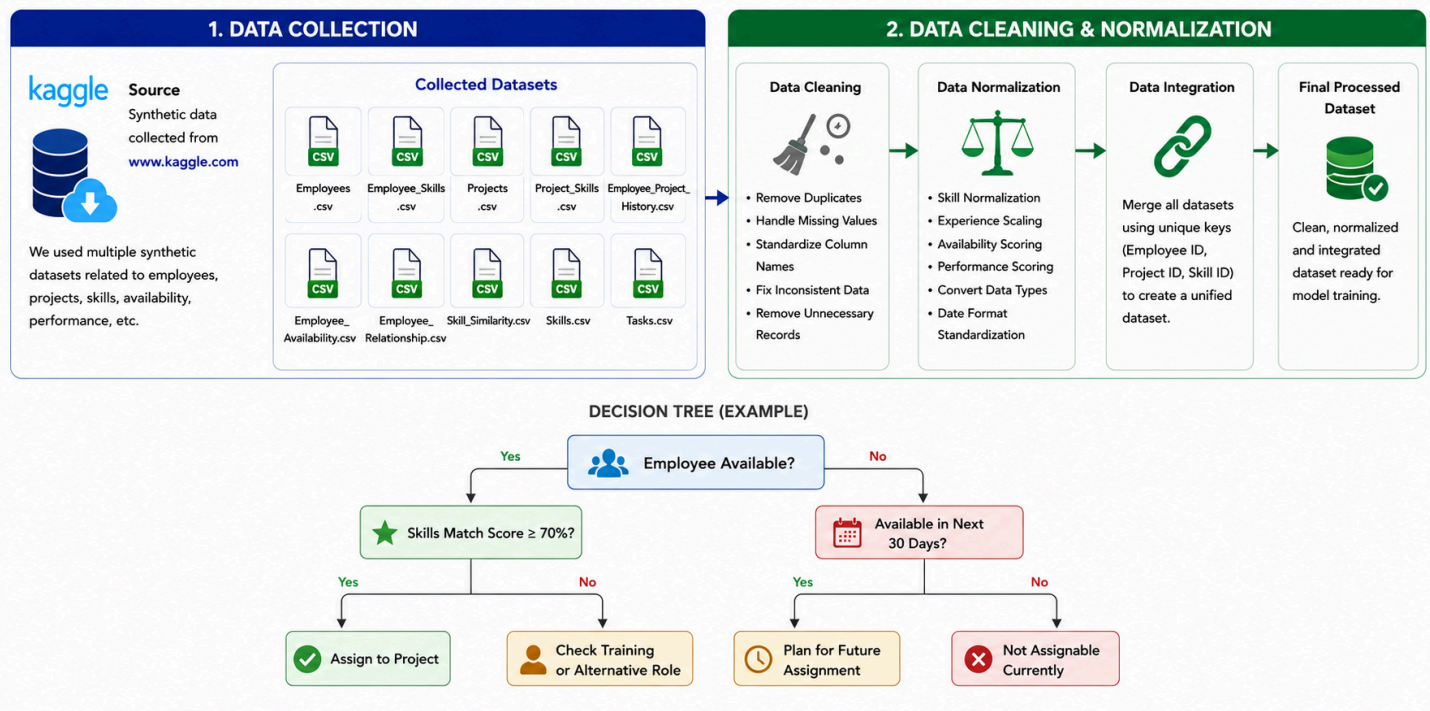


HAF-EPA

DATA COLLECTION & NORMALIZATION PIPELINE



Since these are **synthetic datasets (not real employee data)** collected from Kaggle, you can explain this section like this:

1. Data Collection

For this project, we used **synthetic datasets collected from Kaggle**. These datasets were artificially generated to simulate real-world employee and project management scenarios while protecting privacy and avoiding the use of sensitive personal information.

The collected datasets include:

- **Employees.csv** – Employee information
- **Employee_Skills.csv** – Employee skill profiles
- **Projects.csv** – Project details
- **Project_Skills.csv** – Skills required for projects
- **Employee_Project_History.csv** – Previous project assignments
- **Employee_Availability.csv** – Employee availability information
- **Employee_Relationship.csv** – Team collaboration relationships
- **Skill_Similarity.csv** – Similarity between different skills
- **Skills.csv** – Master skill database
- **Tasks.csv** – Project task information

These datasets were used as the foundation for training and evaluating the employee recommendation system

2. Data Cleaning & Normalization

After collecting the synthetic datasets, several preprocessing steps were performed:

- Removed duplicate records
- Handled missing values
- Standardized column names and data formats
- Normalized skill, experience, availability, and performance-related features
- Merged all datasets using common identifiers such as Employee ID, Project ID, and Skill ID

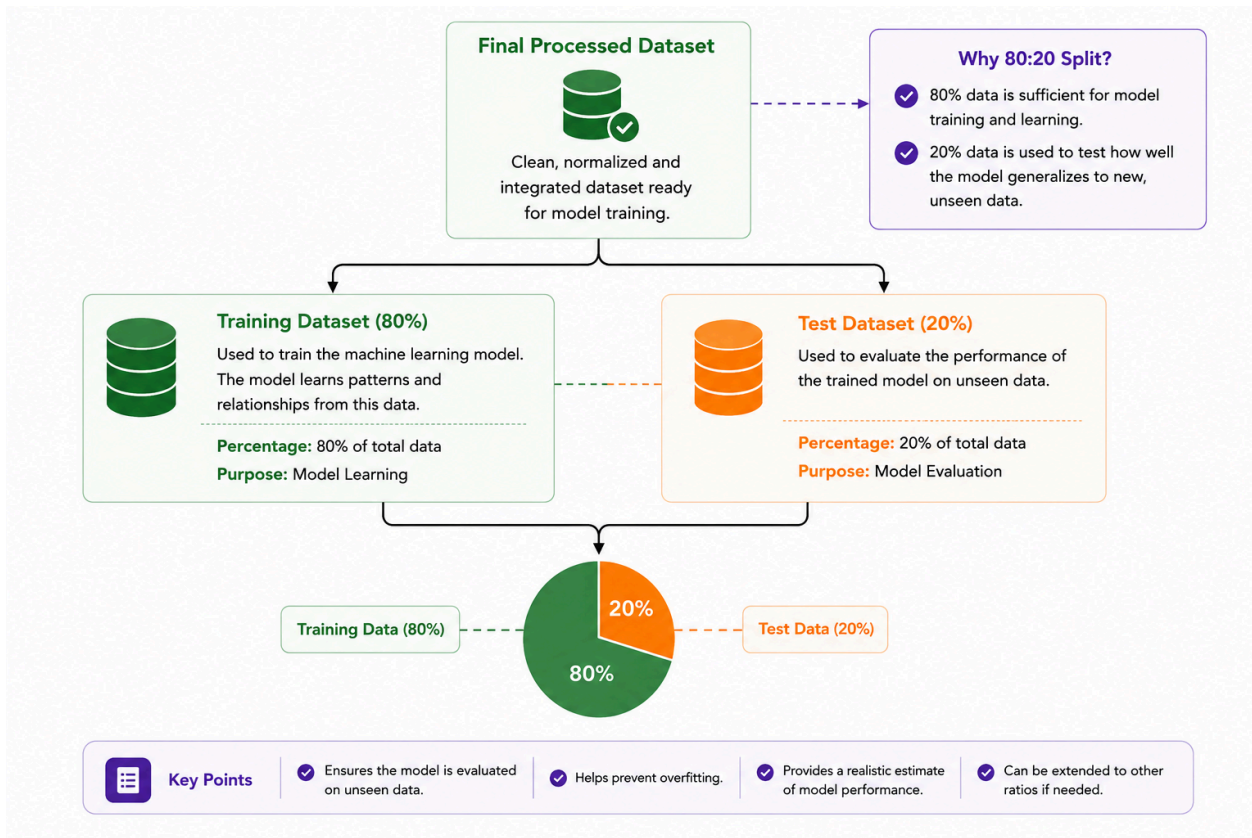
The result was a **clean, normalized, and integrated dataset** ready for machine learning model training.

3. Decision Tree Logic

A simple decision-making logic was designed to evaluate employee suitability:

1. Check employee availability.
2. If available, evaluate skill match score.
3. If the skill match score is **70% or higher**, recommend the employee for project assignment.
4. If the score is lower, suggest training or an alternative role.
5. If the employee is unavailable, check future availability.
6. Based on future availability, either plan a future assignment or mark the employee as currently not assignable.

This logic helps ensure that project recommendations consider both employee skills and availability.



3. Dataset Splitting (Training & Testing)

After data preprocessing, the final dataset was divided into two parts using an **80:20 ratio**.

Training Dataset (80%)

- Used to train the machine learning model.
- The model learns patterns, relationships, and employee-project matching behavior from this data.
- Helps the model understand how different features influence recommendations.

Test Dataset (20%)

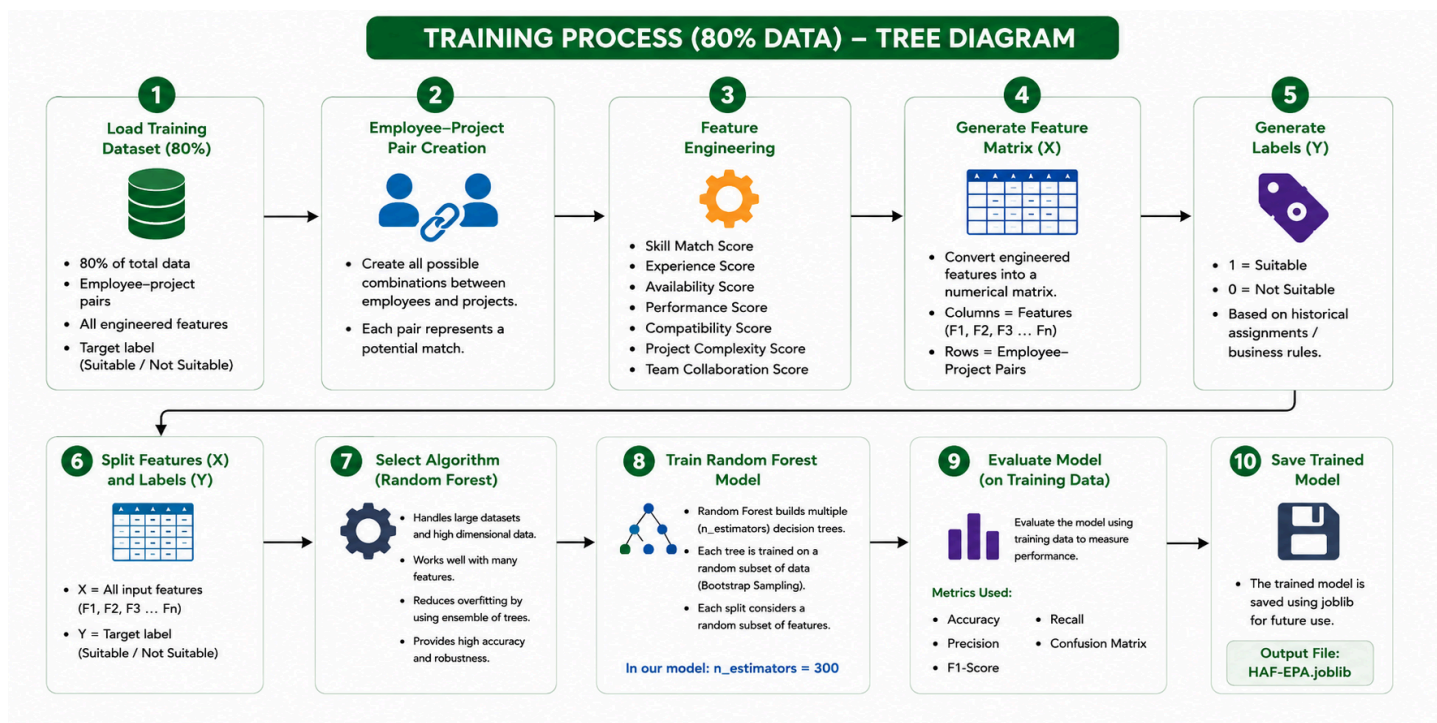
- Used to evaluate the trained model.
- Contains unseen data that was not used during training.
- Measures how well the model performs on new employee and project scenarios.

Why 80:20 Split?

- **80%** provides enough data for effective model learning.
- **20%** provides a reliable evaluation of model performance.
- Helps prevent overfitting.
- Ensures the model can generalize to unseen data.

Outcome:

The trained model was tested using the 20% test dataset, and the evaluation results were used to verify the accuracy and reliability of the employee recommendation system.



4. Model Training Process (Training Dataset – 80%)

The **80% training dataset** is used to build and train the employee recommendation model.

Step 1: Load Training Dataset

The training dataset contains employee-project pairs, engineered features, and target labels (Suitable / Not Suitable).

Step 2: Employee–Project Pair Creation

All possible employee-project combinations are generated. Each combination represents a potential employee assignment for a project.

Step 3: Feature Engineering

Relevant features are created to describe the suitability of an employee for a project, including:

- Skill Match Score
- Experience Score
- Availability Score
- Performance Score
- Compatibility Score
- Project Complexity Score
- Team Collaboration Score

Step 4: Generate Feature Matrix (X)

The engineered features are converted into a numerical feature matrix, where:

- Rows represent employee-project pairs.
- Columns represent feature values.

Step 5: Generate Labels (Y)

Target labels are created based on historical assignments and business rules:

- **1 = Suitable**
- **0 = Not Suitable**

Step 6: Split Features and Labels

The dataset is separated into:

- **X (Features)** – Input variables used for learning.
- **Y (Labels)** – Target outcomes to predict.

Step 7: Select Algorithm

A Random Forest Classifier is selected because it:

- Handles large datasets effectively.
- Works well with multiple features.
- Reduces overfitting through ensemble learning.
- Provides robust and accurate predictions.

Step 8: Train the Random Forest Model

The model is trained using multiple decision trees. Each tree learns from different subsets of data and features, and their combined predictions improve overall performance.

Model Configuration:

⇒ Number of Trees (n_estimators): **300**

Step 9: Evaluate the Model

⇒ The trained model is evaluated using standard machine learning metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

These metrics help measure the effectiveness of the recommendation system.

Step 10: Save the Trained Model

After training and evaluation, the final model is saved using Joblib for future predictions.

Output File: `HAF-EPA.joblib`

CROSS VALIDATION (K-FOLD) – TREE DIAGRAM



We use K-Fold Cross Validation to evaluate the model on training data and ensure better generalization.

1



Training Dataset (80%)

Total training data is used for cross validation.

2



Split into K Folds

The training data is divided into K equal folds.
In our model: K = 5

Fold 1

Fold 2

Fold 3

Fold 4

Fold 5

3



Cross Validation Process (K = 5)

In each iteration, one fold is used for validation and the remaining (K-1) folds are used for training.

Iteration	Training Folds (K-1)	Validation Fold (1 Fold)	Result (Metric)
Iteration 1	Fold 2, 3, 4, 5	Fold 1	Score 1
Iteration 2	Fold 1, 3, 4, 5	Fold 2	Score 2
Iteration 3	Fold 1, 2, 4, 5	Fold 3	Score 3
Iteration 4	Fold 1, 2, 3, 5	Fold 4	Score 4
Iteration 5	Fold 1, 2, 3, 4	Fold 5	Score 5

4



Calculate Average Score

The performance metric is calculated for each iteration and then averaged.

Average Score

$$= (\text{Score 1} + \text{Score 2} + \text{Score 3} + \text{Score 4} + \text{Score 5}) / 5$$

5

Use Average Score

This average score is used to evaluate the model and select the best hyperparameters.



MODEL EVALUATION & RECOMMENDATION GENERATION



EMPLOYEE REFERENCE GENERATION PIPELINE

1



Recommendation Results

Use the ranked recommendation results from the test dataset.



2



Select Top 200 Employees

Select the top 200 employees based on their predicted suitability scores.



3



Build Employee Summary

Create a complete summary for each selected employee using multiple data sources.



Skills



Experience



Availability



Performance



Relationship
Score



4



Generate Top Employee Summary Excel

Save detailed information of top 200 employees for review and analysis.

HAF-EPA_top_200_employees.xlsx



5



Generate Employee Reference CSV

Create a clean reference file used by the web application for fast lookups.

HAF-EPA_employee_reference.csv



6



Used in Web Application

The reference CSV is used by the web application to recommend the most suitable employees for new projects.